

Build Your First REST API with Flask

Comprehensive Lesson Guide & Code Examples (HTTP Methods, Dynamic Routes, Data Processing)

1. What is an API? Meet Waiter Flask-y!

An API (Application Programming Interface) acts as a messenger between different applications or systems. Think of ordering your favorite snack at a Dhaba:

1. The Client (YOU)	2. The API (Waiter Flask-y)	3. The Server (The Kitchen)
The customer placing the order for snacks and drinks.	Takes your request/order to the kitchen and brings your data back.	Where your data is prepared, processed, and stored.

2. Communication Format: JSON

JSON (JavaScript Object Notation) is the standardized format that both clients and server computers understand effortlessly. It structures data in key-value pairs. In Flask, we use the built-in `jsonify()` helper function to convert Python dictionaries or lists into valid JSON HTTP responses.

3. Comprehensive Code Examples

Example 1: Basic GET Route returning JSON

This is the starting point for building REST APIs in Flask. It maps a URL path (e.g. `/menu`) to a function that returns formatted JSON data.

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/menu', methods=['GET'])
def get_menu():
    indian_menu = {
        "snack": "Samosa",
        "drink": "Masala Chai",
        "sweet": "Gulab Jamun"
    }
    return jsonify(indian_menu)

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

Example 2: Dynamic URL Parameters (GET Specific Item)

APIs often need to deliver dynamic resource details based on unique IDs or keys supplied directly within the URL route.

```

from flask import Flask, jsonify

app = Flask(__name__)

# Sample database dictionary
items_db = {
    1: {"name": "Samosa", "price": 20, "category": "Snack"},
    2: {"name": "Masala Chai", "price": 15, "category": "Drink"},
    3: {"name": "Gulab Jamun", "price": 30, "category": "Sweet"}
}

@app.route('/menu/', methods=['GET'])
def get_item(item_id):
    item = items_db.get(item_id)
    if item:
        return jsonify({"success": True, "item": item}), 200
    else:
        return jsonify({"success": False, "error": "Item not found"}), 404

if __name__ == '__main__':
    app.run(debug=True)

```

Example 3: Handling POST Requests (Adding New Data via JSON Body)

To accept data sent by client applications (such as submitting a form or adding an order), we use `request.get_json()` with the POST method.

```

from flask import Flask, jsonify, request

app = Flask(__name__)

menu_list = [
    {"id": 1, "name": "Samosa", "price": 20}
]

@app.route('/menu', methods=['POST'])
def add_item():
    data = request.get_json()

    if not data or 'name' not in data or 'price' not in data:
        return jsonify({"error": "Invalid payload. 'name' and 'price' required."}), 400

    new_item = {
        "id": len(menu_list) + 1,
        "name": data['name'],
        "price": data['price']
    }
    menu_list.append(new_item)
    return jsonify({"message": "Item added successfully", "item": new_item}), 201

if __name__ == '__main__':
    app.run(debug=True)

```

Example 4: Full CRUD Operations (GET, POST, PUT, DELETE)

A complete RESTful service manages data operations through standard HTTP verbs: GET (Read), POST (Create), PUT (Update), and DELETE (Remove).

```

from flask import Flask, jsonify, request

app = Flask(__name__)

orders = [
    {"id": 101, "item": "Masala Chai", "quantity": 2, "status": "Pending"}
]

# 1. READ ALL ORDERS
@app.route('/orders', methods=['GET'])
def get_orders():
    return jsonify({"orders": orders, "total": len(orders)}), 200

# 2. UPDATE AN ORDER (PUT)
@app.route('/orders/', methods=['PUT'])
def update_order(order_id):
    data = request.get_json()
    for order in orders:
        if order["id"] == order_id:
            order["status"] = data.get("status", order["status"])
            order["quantity"] = data.get("quantity", order["quantity"])
            return jsonify({"message": "Order updated successfully", "order": order}), 200
    return jsonify({"error": "Order ID not found"}), 404

# 3. DELETE AN ORDER
@app.route('/orders/', methods=['DELETE'])
def delete_order(order_id):
    global orders
    initial_len = len(orders)
    orders = [o for o in orders if o["id"] != order_id]
    if len(orders) < initial_len:
        return jsonify({"message": f"Order {order_id} deleted successfully"}), 200
    return jsonify({"error": "Order ID not found"}), 404

if __name__ == '__main__':
    app.run(debug=True)

```

4. Testing Your Flask APIs

You can test your GET endpoints directly in any web browser. For POST, PUT, and DELETE endpoints, use tools like **Postman**, **cURL**, or VS Code REST Client extensions by specifying headers (Content-Type: application/json) and sending a raw JSON body payload.

5. Student Practice & Review

Question 1 (Multiple Choice)

In the restaurant analogy, what does the API represent?

- (A) The customer placing the order
- (B) The kitchen preparing the food
- (C) The waiter carrying orders and delivering food back
- (D) The dining table

Question 2 (Multiple Choice)

Which Flask helper function is used to convert a Python dictionary into a JSON HTTP response?

- (A) `json.dumps()`
- (B) `jsonify()`
- (C) `convert_json()`
- (D) `render_template()`

Question 3 (Coding Challenge)

Write a Flask route for `/my-team` that returns a JSON response containing key-value pairs for: `team_name`, `captain`, `trophies_won`, and `top_batsman`.

Question 4 (Advanced Coding Challenge)

Write a Flask POST route `/api/discount` that receives a JSON payload containing `{"price": 100, "discount_percent": 15}`, calculates the discounted price, and returns a JSON response with the calculated original price, discount amount, and final total.